

Отчет по лабораторной работе №14

Архитектура компьютера

Агапова Анна Антоновна

Содержание

2	Задание.....	6
3	Выполнение лабораторной работы.....	7
4	Выводы	13
	Ответы на контрольные вопросы.....	14

Список иллюстраций

3.1	Создание файла	7
3.2	Код.....	8
3.3	Результат	8
3.4	Создание файла	9
3.5	Код	9
3.6	Результат.....	10
3.7	Создание файла.....	11
3.8	Код.....	11
3.9	Результат	12

Список таблиц

1 Цель работы

Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

2 Задание

1. Написать командный файл, реализующий упрощённый механизм семафоров. Командный файл должен в течение некоторого времени t_1 дожидаться освобождения ресурса, выдавая об этом сообщение, а дождавшись его освобождения, использовать его в течение некоторого времени $t_2 < t_1$, также выдавая информацию о том, что ресурс используется соответствующим командным файлом (процессом). Запустить командный файл в одном виртуальном терминале в фоновом режиме, перенаправив его вывод в другой ($> /dev/tty\#$, где $\#$ — номер терминала куда перенаправляется вывод), в котором также запущен этот файл, но не фоновом, а в привилегированном режиме. Доработать программу так, чтобы имелась возможность взаимодействия трёх и более процессов.
2. Реализовать команду `man` с помощью командного файла. Изучите содержимое каталога `/usr/share/man/man1`. В нем находятся архивы текстовых файлов, содержащих справку по большинству установленных в системе программ и команд. Каждый архив можно открыть командой `less` сразу же просмотрев содержимое справки. Командный файл должен получать в виде аргумента командной строки название команды и в виде результата выдавать справку об этой команде или сообщение об отсутствии справки, если соответствующего файла нет в каталоге `man1`.
3. Используя встроенную переменную `$RANDOM`, напишите командный файл, генерирующий случайную последовательность букв латинского алфавита. Учтите, что `$RANDOM` выдаёт псевдослучайные числа в диапазоне от 0 до 32767.

3 Выполнение лабораторной работы

1. Создаю исполняемый файл для первого задания. (рис. 3.1).

```
aaagapova@aaagapova:~/tmp$ touch 11.sh  
aaagapova@aaagapova:~/tmp$ chmod +x 11.sh
```

Рис. 3.1: Создание файла

2. Пишу программу. (рис. 3.2).

```
#!/bin/bash

lockfile="./lock.file"
exec {fn}>$lockfile

while test -f "$lockfile"
do
    if flock -n ${fn}
    then
        echo "File is blocked"
        sleep 5
        echo "File is unlocked"
        flock -u ${fn}
    else
        echo "File is blocked"
        sleep 5
    fi
done
```

Рис. 3.2: Код

3. Результат работы программы. (рис. 3.3).

```
aaagapova@aaagapova:~/tmp$ bash 11.sh
File is blocked
File is unlocked
File is blocked
File is unlocked
File is blocked
File is unlocked
```

Рис. 3.3: Результат

4. Создаю исполняемый файл для второго задания. (рис. 3.4).


```
aaaagapova@aaaagapova:~$ touch 12.sh  
aaaagapova@aaaagapova:~$ chmod +x 12.sh
```

Рис. 3.4: Создание файла

5. Пишу программу. (рис. 3.5).

```
#!/bin/bash  
  
a=$1  
  
if test -f "/usr/share/man/man1/${a}.1.gz"  
then  
    less /usr/share/man/man1/${a}.1.gz  
else  
    echo "There is no such command"  
fi
```

Рис. 3.5: Код

6. Результат работы программы. (рис. 3.6).

```
ESC[4mESC[24m(1) User Co
ESC[4mESC[24m(1)
ESC[1mNAMEESC[0m
ls - list directory contents
ESC[1mSYNOPSISESC[0m
ESC[1mls ESC[22m[ESC[4mOPTIONESC[24m]... [ESC[4mFILEESC[24m]...
ESC[1mDESCRIPTIONESC[0m
List information about the FILEs (the current directory by default
of ESC[1m-cftuvSUX ESC[22mnor ESC[1m--sort ESC[22mis specified.
Mandatory arguments to long options are mandatory for short opti
ESC[1m-aESC[22m, ESC[1m--allESC[0m
do not ignore entries starting with .
ESC[1m-AESC[22m, ESC[1m--almost-allESC[0m
do not list implied . and ..
```

Рис. 3.6: Результат

7. Создаю исполняемый файл для третьего задания. (рис. 3.7).

```
aaagapova@aaagapova:~/tmp$ touch 13.sh
aaagapova@aaagapova:~/tmp$ chmod +x 13.sh
```

Рис. 3.7: Создание файла

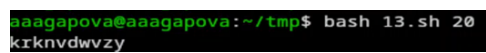
8. Пишу программу. (рис. 3.8).

```
#!/bin/bash

for ((i=0; i<10; i++))
do
    num=$(( ($RANDOM % 26) + 1 ))
    case $num in
        1) echo -n a;;
        2) echo -n b;;
        3) echo -n c;;
        4) echo -n d;;
        5) echo -n e;;
        6) echo -n f;;
        7) echo -n g;;
        8) echo -n h;;
        9) echo -n i;;
        10) echo -n j;;
        11) echo -n k;;
        12) echo -n l;;
        13) echo -n m;;
        14) echo -n n;;
        15) echo -n o;;
        16) echo -n p;;
        17) echo -n r;;
        18) echo -n s;;
        19) echo -n t;;
        20) echo -n q;;
        21) echo -n u;;
        22) echo -n v;;
        23) echo -n w;;
        24) echo -n x;;
        25) echo -n y;;
        26) echo -n z;;
    esac
done
echo
```

Рис. 3.8: Код

9. Результат работы программы. (рис. 3.9).

A terminal window with a black background. The prompt is 'aaagapova@aaagapova:~/tmp\$' in green. The command 'bash 13.sh 20' is entered in green. The output 'krknvdwvzy' is shown in white on the next line.

```
aaagapova@aaagapova:~/tmp$ bash 13.sh 20
krknvdwvzy
```

Рис. 3.9: Результат

4 Выводы

Я изучила основы программирования в оболочке ОС UNIX, научилась писать более сложные командные файлы с использованием логических управляющих инструкций и циклов.

Ответы на контрольные вопросы

1. Ошибка в строке `while [ $1 != "exit" ]` — не хватает пробелов, правильно: `while [ "$1" != "exit" ]`
2. Конкатенация строк — через `VAR3="VAR1VAR2"` или `VAR1+=" World"`
3. Утилита `seq` — генерирует последовательности чисел. Через цикл `for` с фигурными скобками, команда `printf`, команда `echo` с раскрытием фигурных скобок.
4. Выражение `$((10/3))` — результат 3 (целочисленное деление)
5. Отличия `zsh` от `bash` — автодополнение, калькулятор, числа с плавающей запятой и т.д.
6. Синтаксис `for ((a=1; a <= LIMIT; a++))` — верен
7. Для сравнения возьмём язык Python. Преимущества `bash`:
 - Предусмотрен по умолчанию в большинстве дистрибутивов Linux и macOS.
 - Удобное перенаправление ввода/вывода и конвейеры (`>`, `>>`, `<`, `|`).
 - Большое количество встроенных команд для работы с файловой системой.
 - Прямой доступ к системным утилитам без дополнительных библиотек. Недостатки `bash`:
 - Не является языком общего назначения, не подходит для сложных приложений.
 - Мало библиотек по сравнению с Python.
 - Каждая внешняя утилита запускается в отдельном процессе, что замедляет работу скрипта.
 - Скрипты сложно запустить на других ОС (например, Windows) без эмуляторов.
 - Синтаксис менее удобочитаем для больших программ.